Assignment 2

Big Data Engineering

2025-09-28

2025 Spring

Monika Shakya – 25548660

94693– Big Data Engineering

Master of Data Science and Innovation University of Technology of Sydney

Table of Contents

List of Figures

List of Tables

1. Introduction

New York City's taxi system is one of the busiest in the world, serving millions of residents and visitors each year. Yellow taxis have been around since the early 1900s, and in 2013 green taxis were added to improve coverage outside Manhattan. Together, they generate a massive amount of trip data, including pick-up and drop-off details, distances, passenger counts, payment types, and fares.

Taxi fares depend on more than just distance — factors like base fares, surcharges, travel time, and tips also play a role, making prediction less straightforward. In this project, we used NYC taxi trip data from 2014–2024 on Databricks with Apache Spark to explore patterns, engineer features, and build machine learning models to predict the total fare per ride.

Accurate fare predictions have value for many stakeholders: passengers can better estimate trip costs, drivers can anticipate earnings, and city planners can monitor fairness and demand patterns. This project shows how big data tools can turn raw trip records into useful insights and predictive models.

2. Dataset

1) Description

The datasets used in this project are publicly released by the **NYC Taxi and Limousine Commission (TLC)** and cover millions of yellow and green taxi trips from **2014–2025**. Each record represents a single trip with details on timing, locations, passengers, fares, and payments.

Yellow Taxi Trips (TPEP): Provided by vendors (Creative Mobile, Curb, Myle, Helix). Key fields include pickup/drop-off times (tpep_pickup_datetime, tpep_dropoff_datetime), passenger_count, trip_distance, RatecodeID, PULocationID/DOLocationID, and store_and_fwd_flag. Payment details cover payment_type, fare_amount, extra, mta_tax, tolls_amount, tip_amount, surcharges (improvement, congestion, airport, cbd_congestion_fee from 2025), and total_amount.

Green Taxi Trips (LPEP): Provided by vendors (Creative Mobile, Curb, Myle). Similar to yellow taxis, but with lpep_pickup_datetime, lpep_dropoff_datetime, and an additional trip_type field (street-hail or dispatch). Green taxis do not include airport_fee but follow the same fare structure, including taxes and surcharges.

Combined Dataset Insights: Both datasets share common fields, enabling unified analysis. Yellow taxis dominate trips in Manhattan and airports, while green taxis were introduced in 2013 to expand service outside Manhattan. Both datasets provide a full fare breakdown, making them well-suited for analyzing pricing patterns and predicting total fares.

3. Data Ingestion and Cleaning

We loaded the data as instructed and stored it in Parquet files—one for green taxis and another for yellow taxis. Due to the large size of the datasets, the saving process on Databricks took a considerable amount of time. After counting the records, we found **83,484,688** entries for green taxis and **907,982,776** entries for yellow taxis. From the initial exploration, it was clear that the yellow taxi data dominates the dataset, which is expected since green taxis were only introduced in 2013, while yellow taxis have been operating in New York City for decades.

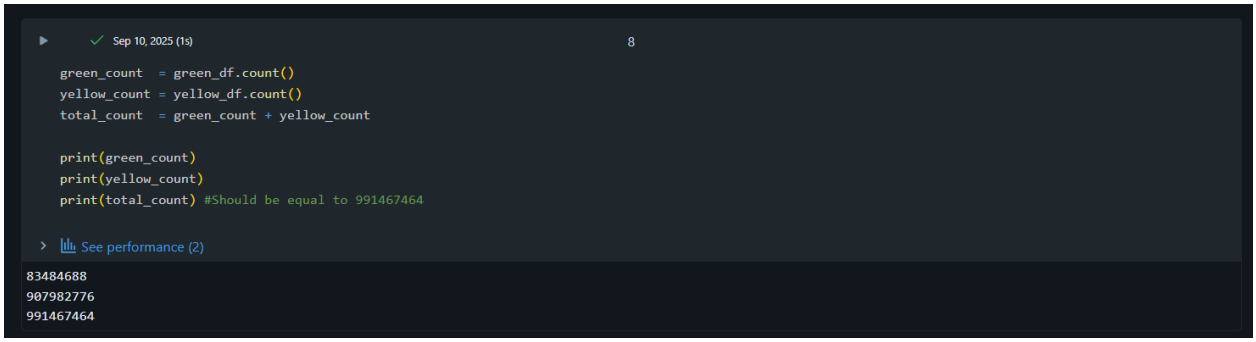

Figure 3. 1 Counting the number of green and yellow taxi trips in Spark. The total count (99,146,764) matches the expected dataset size, confirming correct ingestion.

As part of the data preparation process, we organized the datasets into Databricks **Volumes** to ensure efficient storage and accessibility. We created a dedicated volume under the catalog workspace, schema bde, and volume name assignment2. Within this volume, we stored the parquet files separately for each dataset:

- **Green Taxi Data** → /Volumes/workspace/bde/assignment2/green
- **Yellow Taxi Data** → /Volumes/workspace/bde/assignment2/yellow

Storing the data in Volumes provides a structured and persistent location within the Databricks environment, making it easier to manage, query, and reuse the cleaned datasets throughout the assignment.

1) Data Cleaning

To ensure the taxi datasets were reliable and suitable for analysis, we applied seven cleaning rules. Each addressed a specific type of error or unrealistic value:

1. **Drop-off before pickup** – Removed trips where the drop-off time was earlier than the pickup time, as these indicate invalid or corrupted timestamps.
2. **Out-of-range/null timestamps** – Removed records with missing or unrealistic dates (outside 2014–2024), to ensure trips fall within valid time boundaries.
3. **Negative speeds** – Dropped trips with negative calculated speeds, which are logically impossible and likely due to meter or timestamp errors.
4. **Unrealistically high speeds** – Removed trips with speeds above 150 km/h, since such values are implausible in NYC traffic and usually caused by faulty data.
5. **Unrealistic durations** – Filtered out trips lasting less than 1 minute or more than 6 hours, as these are unlikely to represent valid taxi rides.
6. **Unrealistic distances** – Removed trips shorter than 0.2 km (accidental flag drops or meter errors) or longer than 200 km (unrealistic for city trips).
7. **Unrealistically low fares** – Dropped trips with fares below $3.30 (less than the minimum NYC taxi flag drop rate), except for special cases such as no-charge, disputed, or voided trips.

By systematically applying these rules, we eliminated invalid, extreme, or corrupted records, ensuring the cleaned dataset reflects **realistic taxi operations** and is suitable for meaningful analysis and predictive modelling. We combined the tables

Table 3. 1 Records for Yellow taxi

Step	Cleaning Rule	Before → After	Dropped	% Dropped
1	Invalid timestamps – drop-off before pick-up	907,982,776 → 907,888,752	94,024	0.01%
2	Negative speeds	907,888,752 → 907,877,308	11,444	0.00%
3	Out-of-range/null timestamps (2014–2024)	907,877,308 → 907,785,926	91,382	0.01%
4	Max speed > 150 km/h	907,785,926 → 906,860,751	925,175	0.10%
5	Unrealistic durations (<1 min or >6 hrs)	906,860,751 → 898,095,383	8,765,370	0.97%
6	Unrealistic distances (<0.2 km or >200 km)	898,095,383 → 894,090,489	4,004,894	0.45%
7	Unreasonably low fares (< $3.30, unless cancelled/disputed/voided)	894,090,489 → 893,609,558	480,931	0.05%
Final Total	After all cleaning steps	893,609,558	14,373,218	1.58%

Table 3. 2 Records for Green taxi

Step	Cleaning Rule	Before → After	Dropped	% Dropped
1	Invalid timestamps – drop-off before pick-up	83,484,688 → 83,483,646	1,042	0.00%
2	Negative speeds	83,483,646 → 83,464,120	19,526	0.02%
3	Out-of-range/null timestamps (2000–2025)	83,464,120 → 83,461,786	2,334	0.00%
4	Max speed > 150 km/h	83,461,786 → 83,286,965	174,821	0.21%
5	Unrealistic durations (<1 min or >6 hrs)	83,286,965 → 81,635,465	1,651,500	1.98%
6	Unrealistic distances (<0.2 km or >200 km)	81,635,465 → 80,921,260	714,205	0.87%
7	Unreasonably low fares (< $3.30, unless cancelled/disputed/voided)	80,921,260 → 80,760,890	160,370	0.20%
Final Total	After all cleaning steps	80,760,890	2,723,798	3.26%

2) Data Integration

Since the yellow and green taxi datasets had slightly different schema structures, we first aligned their columns to ensure compatibility before merging them into a single dataset.

- For the **yellow taxi data**, the fields ehail_fee and trip_type were missing, so we added them with default values of 0. A new column, color = "yellow", was also introduced to identify the source dataset.
- For the **green taxi data**, the field airport_fee was missing, so we added it with a default value of 0. Similarly, a color = "green" column was introduced.

After aligning both schemas, the datasets were combined using a **union operation**, and the resulting table was validated by:

- Checking row counts for yellow, green, and combined datasets.
- Inspecting the schema to confirm that all expected columns were present.

This process produced a single unified dataset (trips_union), containing both yellow and green taxi trips in a consistent format, ready for further analysis and modelling.

3) Geographic Enrichment with Taxi Zones

To add geographic context, the trip dataset was enriched with the official **TLC Taxi Zone Lookup Table**, which links each LocationID to its borough, zone, and service zone. The enrichment followed three main steps. First, the zone lookup file was loaded into Databricks, where IDs were cast to integers and text fields cleaned for consistency. Second, two alias tables were created to avoid column conflicts: a pickup version (pu_borough, pu_zone, pu_service_zone) and a drop-off version (do_borough, do_zone, do_service_zone). Finally, these were joined with the trips dataset on PULocationID and DOLocationID using broadcast joins for efficiency, since the lookup table is relatively small.

The result was an enriched dataset (trips_enriched) that included pickup and drop-off details for every trip. It was stored in Delta format under workspace.bde.trips_final, ensuring schema enforcement, efficient I/O, and versioning. After all cleaning and integration, the final dataset contained **974,457,767 rows**, which became the foundation for subsequent EDA and machine learning tasks.

4. Business Questions

i. For each year and month, what was the total number of trips?

Query:

```
SELECT
  date_trunc('month', pickup_ts)     AS month_start,
  date_format(pickup_ts, 'yyyy-MM')  AS month_ym,
  date_format(pickup_ts, 'MMM yyyy') AS month_label,
  COUNT(*)                           AS total_trips
FROM workspace.bde.trips_final
WHERE pickup_ts IS NOT NULL
GROUP BY 1,2,3
ORDER BY month_start;
```

Figure 4. 1 Query for total number of trips

Output:

	month_start	month_ym	month_label	total_trips
1	2014-01-01T00:00:00.000+00:...	2014-01	Jan 2014	14408562
2	2014-02-01T00:00:00.000+00:...	2014-02	Feb 2014	13896457
3	2014-03-01T00:00:00.000+00:...	2014-03	Mar 2014	16520241
4	2014-04-01T00:00:00.000+00:...	2014-04	Apr 2014	15745966
5	2014-05-01T00:00:00.000+00:...	2014-05	May 2014	16006003
6	2014-06-01T00:00:00.000+00:...	2014-06	Jun 2014	14969591
7	2014-07-01T00:00:00.000+00:...	2014-07	Jul 2014	14208319
8	2014-08-01T00:00:00.000+00:...	2014-08	Aug 2014	13692386
9	2014-09-01T00:00:00.000+00:...	2014-09	Sep 2014	14545496
10	2014-10-01T00:00:00.000+00:...	2014-10	Oct 2014	15606306
11	2014-11-01T00:00:00.000+00:...	2014-11	Nov 2014	14656165
12	2014-12-01T00:00:00.000+00:...	2014-12	Dec 2014	14551406

Figure 4. 2 Sample output for total number of trips

The results show monthly trip volumes from 2014 to 2024. In 2014, the average was around **1.4 million trips per month**, with peaks in **March and May at about 1.6 million trips**. This indicates clear seasonal demand patterns, with higher volumes in spring and early summer. To address these fluctuations, taxi operators should **adjust driver scheduling** to increase availability during peak months and apply **dynamic pricing or targeted promotions** in quieter periods to better balance supply and demand.

ii. For each year and month, which day of had the most trips?

Query:

```
WITH dow AS (
  SELECT
    date_trunc('month', pickup_ts)     AS month_start,
    date_format(pickup_ts, 'yyyy-MM')  AS month_ym,
    date_format(pickup_ts, 'MMM yyyy') AS month_label,
    date_format(pickup_ts, 'EEEE')     AS dow_name,
    COUNT(*)                           AS trips
  FROM workspace.bde.trips_final
  WHERE pickup_ts IS NOT NULL
  GROUP BY 1,2,3,4
)
SELECT
  month_label,
  month_ym,
  dow_name      AS top_day_of_week,
  trips         AS trips_on_top_dow
FROM (
  SELECT
    d.*,
    ROW_NUMBER() OVER (PARTITION BY month_start ORDER BY trips DESC, dow_name ASC) AS rn
  FROM dow d
)
WHERE rn = 1
ORDER BY month_ym;
```

Figure 4. 3 Query for day with maximum trips

Output:

	month_label	month_ym	top_day_of_week	trips_on_top_dow
1	Jan 2014	2014-01	Friday	2424757
2	Feb 2014	2014-02	Saturday	2248509
3	Mar 2014	2014-03	Saturday	2992731
4	Apr 2014	2014-04	Wednesday	2635980
5	May 2014	2014-05	Friday	2786539
6	Jun 2014	2014-06	Sunday	2326828
7	Jul 2014	2014-07	Thursday	2421360
8	Aug 2014	2014-08	Friday	2353740
9	Sep 2014	2014-09	Tuesday	2304863
10	Oct 2014	2014-10	Friday	2712730
11	Nov 2014	2014-11	Saturday	2809992
12	Dec 2014	2014-12	Wednesday	2347345

Figure 4. 4 Output for day with maximum trips

The analysis for sample 2014 shows that Fridays and Saturdays are often the busiest days, reflecting strong weekend demand, while some months also peak midweek. Fleet planning can be done accordingly on busy days to meet the high demand weeks for better transportation.

iii. For each year and month, which hour of the day had the most trips?

Query:

```
WITH hr AS (
  SELECT
    date_trunc('month', pickup_ts)     AS month_start,
    date_format(pickup_ts, 'yyyy-MM')  AS month_ym,
    date_format(pickup_ts, 'MMM yyyy') AS month_label,
    hour(pickup_ts)                    AS hour_of_day,
    COUNT(*)                           AS trips
  FROM workspace.bde.trips_final
  WHERE pickup_ts IS NOT NULL
  GROUP BY 1,2,3,4
)
SELECT
  month_label,
  month_ym,
  hour_of_day  AS top_hour_of_day,
  trips        AS trips_in_top_hour
FROM (
  SELECT
    h.*,
    ROW_NUMBER() OVER (PARTITION BY month_start ORDER BY trips DESC, hour_of_day ASC) AS rn
  FROM hr h
)
WHERE rn = 1
ORDER BY month_ym;
```

Figure 4. 5 Query for hr of the day with maximum trips

Output:

	month_label	month_ym	top_hour_of_day	trips_in_top_hour
1	Jan 2014	2014-01	19	917570
2	Feb 2014	2014-02	19	897852
3	Mar 2014	2014-03	19	1055184
4	Apr 2014	2014-04	19	996208
5	May 2014	2014-05	19	1005732
6	Jun 2014	2014-06	19	924107
7	Jul 2014	2014-07	19	904094
8	Aug 2014	2014-08	19	848554
9	Sep 2014	2014-09	19	919198
10	Oct 2014	2014-10	19	997236
11	Nov 2014	2014-11	19	906263
12	Dec 2014	2014-12	19	890586

Figure 4. 6 Output for hr of day with maximum trips

The results highlight the hour of the day with the highest trip counts for each month. Example in 2014, across all months, the busiest hour was consistently 7 PM (19:00). This strong pattern indicates that early evening hours are the peak demand period for taxi services in New York City, likely to reflect commuters returning home, evening leisure activities, and nightlife-related travel understanding this the planning can be done by the taxi drivers.

iv. For each year and month, what was the average number of passengers?

Query:

```
SELECT
  date_trunc('month', pickup_ts)      AS month_start,
  date_format(pickup_ts, 'yyyy-MM')   AS month_ym,
  date_format(pickup_ts, 'MMM yyyy')  AS month_label,
  ROUND(AVG(passenger_count), 2)      AS avg_passengers_per_trip
FROM workspace.bde.trips_final
WHERE pickup_ts IS NOT NULL
GROUP BY 1,2,3
ORDER BY month_start;
```

Figure 4. 7 Query for average number of passengers

Output:

	month_start	month_ym	month_label	avg_passengers_per_trip
1	2014-01-01T00:00:00.000+00:...	2014-01	Jan 2014	1.69
2	2014-02-01T00:00:00.000+00:...	2014-02	Feb 2014	1.68
3	2014-03-01T00:00:00.000+00:...	2014-03	Mar 2014	1.68
4	2014-04-01T00:00:00.000+00:...	2014-04	Apr 2014	1.68
5	2014-05-01T00:00:00.000+00:...	2014-05	May 2014	1.68
6	2014-06-01T00:00:00.000+00:...	2014-06	Jun 2014	1.68
7	2014-07-01T00:00:00.000+00:...	2014-07	Jul 2014	1.68
8	2014-08-01T00:00:00.000+00:...	2014-08	Aug 2014	1.68
9	2014-09-01T00:00:00.000+00:...	2014-09	Sep 2014	1.66
10	2014-10-01T00:00:00.000+00:...	2014-10	Oct 2014	1.66
11	2014-11-01T00:00:00.000+00:...	2014-11	Nov 2014	1.66
12	2014-12-01T00:00:00.000+00:...	2014-12	Dec 2014	1.66

Figure 4. 8 Output for average number of passengers

On average, trips carried between 1.66 and 1.69 passengers across the year 2014. This indicates that most trips were single-passenger rides, with only minor fluctuations throughout the year 2014**.** This shows that the driver should focus on single-passenger rides.

v. For each year and month, what was the average amount paid per trip?

Query:

```
SELECT
  date_trunc('month', pickup_ts)     AS month_start,
  date_format(pickup_ts, 'yyyy-MM')  AS month_ym,
  date_format(pickup_ts, 'MMM yyyy') AS month_label,
  ROUND(AVG(total_amount), 2)        AS avg_amount_per_trip
FROM workspace.bde.trips_final
WHERE pickup_ts IS NOT NULL
GROUP BY 1,2,3
ORDER BY month_start;
```

Figure 4. 9 Query for average amount paid per trip

Output:

	month_start	month_ym	month_label	avg_amount_per_trip
1	2014-01-01T00:00:00.000+00:...	2014-01	Jan 2014	14.39
2	2014-02-01T00:00:00.000+00:...	2014-02	Feb 2014	14.51
3	2014-03-01T00:00:00.000+00:...	2014-03	Mar 2014	14.68
4	2014-04-01T00:00:00.000+00:...	2014-04	Apr 2014	14.97
5	2014-05-01T00:00:00.000+00:...	2014-05	May 2014	15.54
6	2014-06-01T00:00:00.000+00:...	2014-06	Jun 2014	15.55
7	2014-07-01T00:00:00.000+00:...	2014-07	Jul 2014	15.21
8	2014-08-01T00:00:00.000+00:...	2014-08	Aug 2014	16.14
9	2014-09-01T00:00:00.000+00:...	2014-09	Sep 2014	15.59
10	2014-10-01T00:00:00.000+00:...	2014-10	Oct 2014	15.59
11	2014-11-01T00:00:00.000+00:...	2014-11	Nov 2014	15.39
12	2014-12-01T00:00:00.000+00:...	2014-12	Dec 2014	15.56

Figure 4. 10 Output for average amount paid per trip

The results show the average fare amount per trip for each month. For example, average fare per trip ranged from around $14.39 in January 2014 to a peak of $16.14 in August 2014. This can give a general understanding of the tip culture as well as revenue for the drivers.

vi. For each year and month, was the average amount paid per passenger?

Query:

```
SELECT
  date_trunc('month', pickup_ts)      AS month_start,
  date_format(pickup_ts, 'yyyy-MM')   AS month_ym,
  date_format(pickup_ts, 'MMM yyyy')  AS month_label,
  ROUND(
    AVG(CASE WHEN passenger_count > 0 THEN total_amount / passenger_count END)
  , 2) AS avg_amount_per_passenger
FROM workspace.bde.trips_final
WHERE pickup_ts IS NOT NULL
GROUP BY 1,2,3
ORDER BY month_start;
```

Figure 4. 11 Query for average amount per passenger

Output:

	month_start	month_ym	month_label	avg_amount_per_passenger
1	2014-01-01T00:00:00.000+00:...	2014-01	Jan 2014	11.66
2	2014-02-01T00:00:00.000+00:...	2014-02	Feb 2014	11.82
3	2014-03-01T00:00:00.000+00:...	2014-03	Mar 2014	11.94
4	2014-04-01T00:00:00.000+00:...	2014-04	Apr 2014	12.15
5	2014-05-01T00:00:00.000+00:...	2014-05	May 2014	12.59
6	2014-06-01T00:00:00.000+00:...	2014-06	Jun 2014	12.6
7	2014-07-01T00:00:00.000+00:...	2014-07	Jul 2014	12.31
8	2014-08-01T00:00:00.000+00:...	2014-08	Aug 2014	13.16
9	2014-09-01T00:00:00.000+00:...	2014-09	Sep 2014	12.7
10	2014-10-01T00:00:00.000+00:...	2014-10	Oct 2014	12.72
11	2014-11-01T00:00:00.000+00:...	2014-11	Nov 2014	12.51
12	2014-12-01T00:00:00.000+00:...	2014-12	Dec 2014	12.65

Figure 4. 12 Output for average amount paid per passenger

The results show the average fare amount per passenger for each month. In early 2014, the average ranged from $11.66 in January to $11.94 in March, before gradually increasing through spring and summer.

vii. For each taxi type (yellow and green), what are the key trip characteristics — specifically the average, median, minimum, and maximum values for trip duration (minutes), trip distance (km), and trip speed (km/h)?

Query:

```sql
WITH base AS (
  SELECT
    color,
    CAST(duration_sec / 60.0 AS DOUBLE) AS duration_min,
    distance_km,
    speed_kmh
  FROM workspace.bde.trips_final
  WHERE duration_sec IS NOT NULL
    AND distance_km  IS NOT NULL
    AND speed_kmh    IS NOT NULL
)
SELECT
  color,
  -- Duration (minutes) with 2 decimals
  ROUND(AVG(duration_min), 2)                              AS avg_duration_min,
  ROUND(percentile_approx(duration_min, 0.5, 10000), 2)    AS median_duration_min,
  ROUND(MIN(duration_min), 2)                              AS min_duration_min,
  ROUND(MAX(duration_min), 2)                              AS max_duration_min,

  -- Distance (km)
  ROUND(AVG(distance_km), 2)                               AS avg_distance_km,
  ROUND(percentile_approx(distance_km, 0.5, 10000), 2)     AS median_distance_km,
  ROUND(MIN(distance_km), 2)                               AS min_distance_km,
  ROUND(MAX(distance_km), 2)                               AS max_distance_km,

  -- Speed (km/h)
  ROUND(AVG(speed_kmh), 2)                                 AS avg_speed_kmh,
  ROUND(percentile_approx(speed_kmh, 0.5, 10000), 2)       AS median_speed_kmh,
  ROUND(MIN(speed_kmh), 2)                                 AS min_speed_kmh,
  ROUND(MAX(speed_kmh), 2)                                 AS max_speed_kmh

FROM base
GROUP BY color
ORDER BY color;
```

Figure 4. 13 Query for duration, distance and speed

Output:

	color	avg_duration_min	median_duration_min	min_duration_min	max_duration_min	avg_distance_km	median_distance_km
1	green	13.93	10.68	1	360	4.92	3.17
2	yellow	14.47	11.32	1	360	4.93	2.77

min_distance_km	max_distance_km	avg_speed_kmh	median_speed_kmh	min_speed_kmh	max_speed_kmh
0.21	199.96	20.52	18.59	0.04	150
0.21	199.99	18.93	16.6	0.04	149.99

Figure 4. 14 Output for duration, distance and speed

The results show that yellow taxis have slightly longer trips (avg ~14.5 mins) than green taxis (~13.9 mins), though distances are nearly identical (~4.9 km). Green taxis recorded higher average speeds (~20.5 km/h) compared to yellow (~18.9 km/h). This suggests green taxis, often operating outside Manhattan, tend to travel faster despite similar trip lengths.

viii. For each taxi type (yellow and green), across every **pickup–dropoff borough pair**, **month**, **day of week**, and **hour of day**, what are the key operational and financial metrics — specifically the **total number of trips**, the **average trip distance (km)**, the **average fare per trip**, and the **total fare amount collected**?

Query:

```
CREATE OR REPLACE TABLE workspace.bde.q3_color_borough_stats AS
SELECT
  color,
  pu_borough,
  do_borough,
  date_format(pickup_ts, 'yyyy-MM')  AS month_ym,
  date_format(pickup_ts, 'EEEE')     AS dow_name,
  hour(pickup_ts)                    AS hour_of_day,
  COUNT(*)                           AS total_trips,
  ROUND(AVG(distance_km), 2)         AS avg_distance_km,
  ROUND(AVG(total_amount), 2)        AS avg_amount_per_trip,
  ROUND(SUM(total_amount), 2)        AS total_amount_paid
FROM workspace.bde.trips_final
WHERE pickup_ts IS NOT NULL
  AND pu_borough IS NOT NULL
  AND do_borough IS NOT NULL
GROUP BY
  color, pu_borough, do_borough,
  date_format(pickup_ts, 'yyyy-MM'),
  date_format(pickup_ts, 'EEEE'),
  hour(pickup_ts);
```

Figure 4. 15 Query for Key characteristics

Output:

	color	pu_borough	do_borough	month_ym	dow_name	hour_of_day	total_trips	avg_distance_km	avg_amount_per_trip	total_amount_paid
1	yellow	Manhattan	Brooklyn	2014-11	Thursday	22	6987	9.83	27.08	189222.28
2	yellow	Queens	Queens	2014-11	Thursday	22	1087	8.28	19.59	21297.91
3	yellow	Brooklyn	Queens	2014-11	Thursday	22	155	10.84	25.19	3904.13
4	yellow	Bronx	Brooklyn	2014-11	Thursday	22	1	22.18	43.5	43.5
5	yellow	Manhattan	Bronx	2014-11	Friday	0	742	14.34	30.68	22766.21
6	yellow	Manhattan	Bronx	2014-11	Friday	3	344	15.88	31.77	10928.03
7	yellow	Queens	Manhattan	2014-11	Friday	3	95	11.67	25.77	2448.16
8	yellow	Manhattan	Brooklyn	2014-11	Friday	4	1544	9.69	23.21	35835.61
9	yellow	Queens	Bronx	2014-11	Friday	6	33	23.79	51.46	1698.29

Figure 4. 16 Output for Key characteristics

The results show the average fare amount per passenger for each month. In early 2014, the average ranged from $11.66 in January to $11.94 in March, before gradually increasing through spring and summer.

ix. In 2024, what share of total revenue was generated by the **Top 10 pickup–dropoff borough pairs**, ranked by total fare amount?

Query:

```sql
WITH revenue_by_pair AS (
  SELECT
    pu_borough,
    do_borough,
    ROUND(SUM(total_amount), 2) AS total_revenue
  FROM workspace.bde.trips_final
  WHERE year(pickup_ts) = 2024
    AND pu_borough IS NOT NULL
    AND do_borough IS NOT NULL
  GROUP BY pu_borough, do_borough
),
ranked AS (
  SELECT
    r.*,
    RANK() OVER (ORDER BY total_revenue DESC) AS rk
  FROM revenue_by_pair r
),
total_rev AS (
  SELECT SUM(total_revenue) AS grand_total
  FROM revenue_by_pair
)

SELECT
  pu_borough,
  do_borough,
  total_revenue,
  ROUND(100 * total_revenue / t.grand_total, 2) AS revenue_share_pct
FROM ranked r
CROSS JOIN total_rev t
WHERE rk <= 10
ORDER BY total_revenue DESC;
```

Figure 4. 17 Query for Key characteristics

Output:

	pu_borough	do_borough	total_revenue	revenue_share_pct
1	Manhattan	Manhattan	708143993.19	62.58
2	Queens	Manhattan	172079124.75	15.21
3	Manhattan	Queens	73964802.01	6.54
4	Queens	Brooklyn	37986639.54	3.36
5	Manhattan	Brooklyn	32665361.47	2.89
6	Queens	Queens	32001880.08	2.83
7	Queens	Unknown	14506079.77	1.28
8	Manhattan	EWR	12500186.67	1.1
9	Brooklyn	Brooklyn	7834487.89	0.69
10	Brooklyn	Manhattan	7782905.98	0.69

Figure 4. 18 Output for Key characteristics

The results show the top 10 pickup–dropoff borough pairs by total revenue in 2024. Trips within Manhattan→Manhattan dominate, contributing about 62.6% of total revenue, followed by Queens→Manhattan (15.2%) and Manhattan→Queens (6.5%). Together, these three flows account for

over 84% of revenue, highlighting the central role of Manhattan trips in NYC's taxi economy, while other borough pairs contribute much smaller shares.

x. What do tipping patterns and trip durations reveal about NYC taxi operations? Specifically:

- What percentage of trips involved tips, and within those, what share were tips of at least **$15**?
- How do trips distributed across duration bins (under 5 mins, 5–10 mins, 10–20 mins, 20–30 mins, 30–60 mins, and 60+ mins) differ in terms of **average speed (km/h)** and **average distance per dollar (km per $)**?

Query:

```
WITH base AS (
  SELECT
    *,
    CAST(duration_sec / 60.0 AS DOUBLE) AS duration_min
  FROM workspace.bde.trips_final
  WHERE duration_sec IS NOT NULL
    AND distance_km IS NOT NULL
    AND speed_kmh IS NOT NULL
    AND total_amount > 0                -- avoid div by zero for km/$
),
tips AS (
  SELECT
    COUNT(*)                                AS total_trips,
    SUM(CASE WHEN tip_amount > 0 THEN 1 ELSE 0 END) AS trips_with_tips,
    SUM(CASE WHEN tip_amount >= 15 THEN 1 ELSE 0 END) AS trips_with_tips_15
  FROM base
),
bins AS (
  SELECT
    CASE
      WHEN duration_min < 5 THEN 'Under 5 Mins'
      WHEN duration_min >= 5 AND duration_min < 10 THEN '5–10 Mins'
      WHEN duration_min >= 10 AND duration_min < 20 THEN '10–20 Mins'
   CASE
     WHEN duration_min < 5 THEN 'Under 5 Mins'
     WHEN duration_min >= 5 AND duration_min < 10 THEN '5–10 Mins'
     WHEN duration_min >= 10 AND duration_min < 20 THEN '10–20 Mins'
     WHEN duration_min >= 20 AND duration_min < 30 THEN '20–30 Mins'
     WHEN duration_min >= 30 AND duration_min < 60 THEN '30–60 Mins'
     ELSE 'At least 60 Mins'
   END
)

SELECT
  -- overall tip stats (same for all rows)
  ROUND(100.0 * t.trips_with_tips / t.total_trips, 2)   AS pct_trips_with_tips,
  ROUND(100.0 * t.trips_with_tips_15 / NULLIF(t.trips_with_tips,0), 2) AS pct_tips_ge_15,
  -- bin-level metrics
  b.duration_bin,
  ROUND(b.avg_speed_kmh, 2)         AS avg_speed_kmh,
  ROUND(b.avg_km_per_dollar, 4)     AS avg_km_per_dollar
FROM bins b
CROSS JOIN tips t
ORDER BY
  CASE b.duration_bin
    WHEN 'Under 5 Mins' THEN 1
    WHEN '5–10 Mins' THEN 2
    WHEN '10–20 Mins' THEN 3
    WHEN '20–30 Mins' THEN 4
    WHEN '30–60 Mins' THEN 5
    ELSE 6
  END;
```

Figure 4. 19 Query for tips

Output:

	.00 pct_trips_with_tips	.00 pct_tips_ge_15	A^B_C duration_bin	1.2 avg_speed_kmh	1.2 avg_km_per_dollar
1	63.01	0.83	Under 5 Mins	19.86	0.1627
2	63.01	0.83	5–10 Mins	17.2	0.2093
3	63.01	0.83	10–20 Mins	17.92	0.2554
4	63.01	0.83	20–30 Mins	21.39	0.2999
5	63.01	0.83	30–60 Mins	25.82	0.3539
6	63.01	0.83	At least 60 Mins	22.61	0.4513

Figure 4. 20 Output for tips

Overall, **63% of trips included tips**, but only **0.83% of tipped trips exceeded $15**. When trips are classified into duration bins, we see that **longer trips (60+ minutes)** provide both higher **average speeds** (~22.6 km/h) and better **value per dollar** (~0.45 km per $), compared to very short trips which are slower and less cost-efficient. This suggests that drivers gain more consistent earnings from **longer-duration trips**, while passengers receive greater distance value.

xi. Which duration bin will you advise a taxi driver to target to maximise his income?

Query:

```
WITH base AS (
  SELECT
    *,
    CAST(duration_sec / 60.0 AS DOUBLE) AS duration_min
  FROM workspace.bde.trips_final
  WHERE duration_sec IS NOT NULL
    AND total_amount IS NOT NULL
),
bins AS (
  SELECT
    CASE
      WHEN duration_min < 5 THEN 'Under 5 Mins'
      WHEN duration_min >= 5 AND duration_min < 10 THEN '5–10 Mins'
      WHEN duration_min >= 10 AND duration_min < 20 THEN '10–20 Mins'
      WHEN duration_min >= 20 AND duration_min < 30 THEN '20–30 Mins'
      WHEN duration_min >= 30 AND duration_min < 60 THEN '30–60 Mins'
      ELSE 'At least 60 Mins'
    END AS duration_bin,
    ROUND(AVG(total_amount), 2) AS avg_revenue_per_trip,
    ROUND(AVG(speed_kmh), 2)    AS avg_speed_kmh,
    ROUND(AVG(try_divide(distance_km, total_amount)), 4) AS avg_km_per_dollar
  FROM base
  GROUP BY
    CASE
      WHEN duration_min < 5 THEN 'Under 5 Mins'
      WHEN duration_min >= 5 AND duration_min < 10 THEN '5–10 Mins'
      WHEN duration_min >= 10 AND duration_min < 20 THEN '10–20 Mins'
      WHEN duration_min >= 20 AND duration_min < 30 THEN '20–30 Mins'
      WHEN duration_min >= 30 AND duration_min < 60 THEN '30–60 Mins'
      ELSE 'At least 60 Mins'
    END
)
SELECT *
FROM bins
ORDER BY avg_revenue_per_trip DESC;
```

Figure 4. 21 Query for revenue

Output:

	duration_bin	avg_revenue_per_trip	avg_speed_kmh	avg_km_per_dollar
1	At least 60 Mins	70.88	22.62	0.4497

Figure 4. 22 Output for revenue

Trips lasting 60+ minutes yield the highest revenue and good value per dollar, while 30-60 minute trips also perform strongly. Shorter trips earn significantly less according to the data. Therefore, drivers should prioritize longer trips for maximum income. But also consider shorter trips for faster revenue.

5. Data Preparation for Modelling

The final enriched dataset contained **974 million taxi trips** after cleaning, integration, and enrichment. Working with the full dataset was computationally infeasible, so a multi-stage sampling strategy was applied.

1) Stratified sampling (4% overall):

We first created a stratified sample across months to reduce the dataset size while preserving temporal patterns. An **overall target of 4% (~39M rows)** was chosen. To ensure adequate evaluation data, the last three months (**Oct–Dec 2024**) were oversampled at **3%**, while all other months were sampled at **≈4.01%**. This produced a stratified sample of **38.9M rows**, split into **train (30.9M), validation (7.7M), and test (0.33M)**, with the test set defined strictly by the final three months

2) Down-Sampling Train/Validation (5%)

Although the stratified sample was suitable for Spark analysis, it remained too large for Pandas-based machine learning workflows. Therefore, we applied an additional **5% random down-sample** (with a fixed seed) to the **train and validation sets only**, reducing them to **~1.55M (train)** and **~0.39M (validation)** rows. The **test set was kept intact (0.33M rows)** to preserve evaluation integrity on the full temporal holdout period.

3) Feature selection

For the numerical features, we chose **passenger_count, distance_km, duration_sec, speed_kmh, and hour_of_day** because they capture the main factors that influence taxi fares without leaking information from the target. Distance and duration directly drive the fare, speed gives insight into traffic conditions, passenger count can reflect shared trips, and hour of day captures peak versus off-peak effects.

For the categorical features, we picked **color, pu_borough, do_borough, payment_type, day_of_week, and month** as they represent practical aspects of how and when a trip happens. These variables help capture differences in fare structure (e.g. yellow vs green taxis), trip location, payment patterns, and seasonal or weekly demand trends — all of which are useful for predicting total fare.

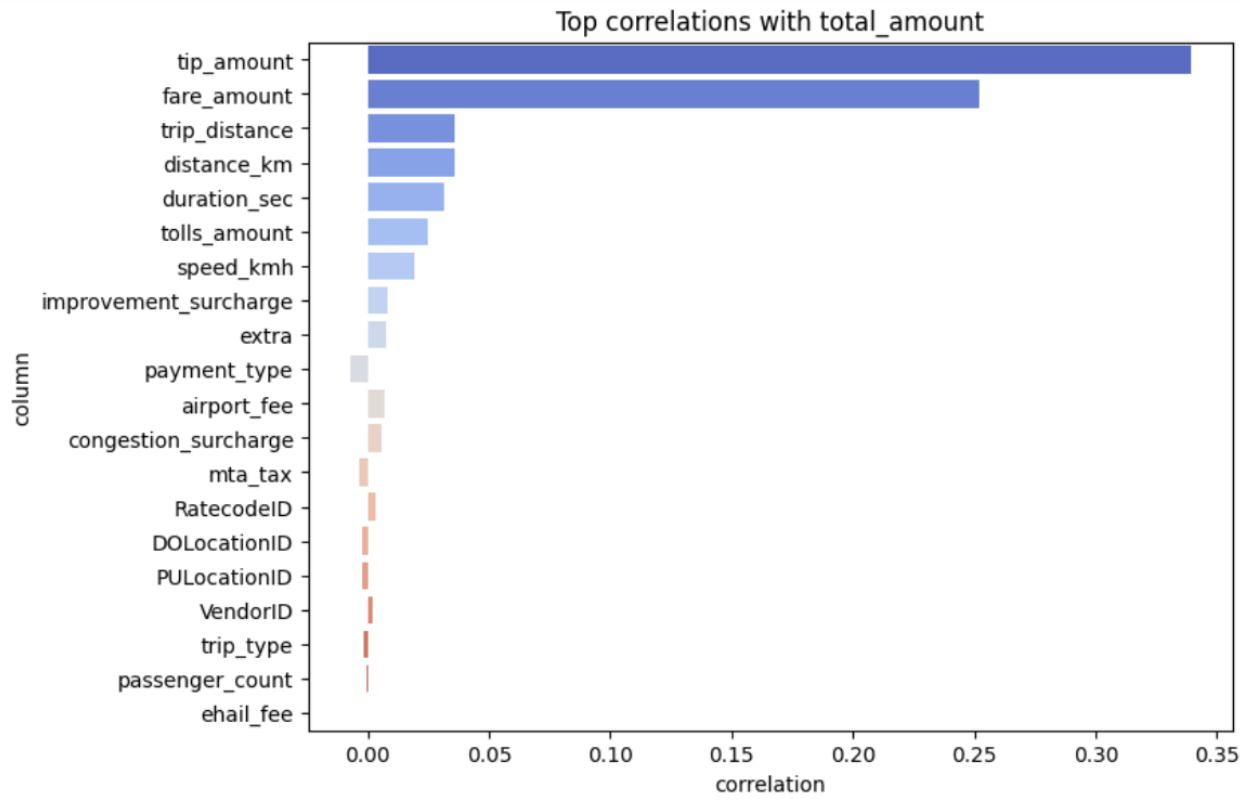

Figure 5. 1 Correlation Between target and features

Numerical features were selected based on their correlation with total_amount, excluding fare components to avoid target leakage.

6. Modelling

1) Base line Modelling

The baseline model predicted fares using the **average total amount per trip**, grouped by taxi color, pickup/drop-off borough, month, weekday, and hour. Missing matches were filled with the **overall color average**.

On the test set (**Oct–Dec 2024**), the baseline achieved an **RMSE of 103.01**, providing a benchmark for evaluating advanced models.

2) Modelling

We experimented with four different regression models: **Linear Regression, Ridge Regression, SGDRegressor (with Huber loss), and a Multi-Layer Perceptron (MLP)**. The dataset was split into training/validation (all months except Oct–Dec 2024) and a held-out test set (Oct–Dec 2024), with RMSE and MAE used for evaluation.

- **Linear Regression** performed reasonably with Val RMSE ≈ **1249** and Test RMSE ≈ **37.1**.
- **Ridge Regression (α=1)** showed slightly worse validation performance and extremely high test error (RMSE > **3500**), indicating poor generalization.
- **SGD with Huber loss** gave unstable results, with Val RMSE ≈ **58k** and Test RMSE ≈ **89k**, making it unsuitable.

- **MLP (64,32 layers)** outperformed all models with Val RMSE ≈ **3.2** and Test RMSE ≈ **13.6**, showing the best fit and lowest prediction error.

For comparison, the **baseline average model** had a Test RMSE of **103.0**. Given the balance of **accuracy and robustness**, the **MLP model** was selected as the best. It consistently achieved much lower RMSE/MAE than the other approaches while remaining computationally feasible, and it clearly outperformed the baseline benchmark.

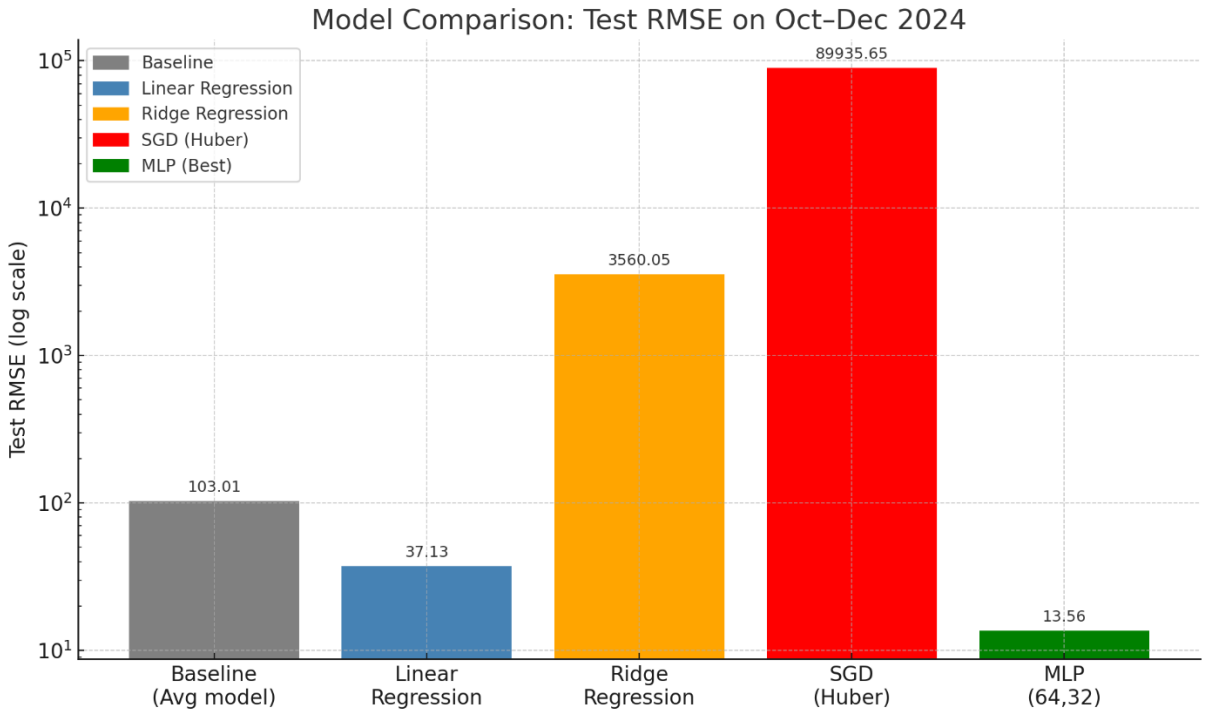

Figure 6. 1 Model comparison

7. Challenges and Model Choices

A key challenge in this project was working within the limits of the **free Databricks edition**, which only allowed 32 GB of memory. This restriction meant we could not train on the full dataset and instead had to carefully sample fractions of the data, ensuring that the sample still represented the overall population. On the serverless cluster, sessions frequently shut down due to memory pressure, and we often had to load data step by step (training first, then validation, then test), which slowed progress.

For modelling, we balanced both **accuracy and practicality**. **Linear** and **Ridge Regression** were chosen first because they train quickly and provide interpretable baselines. **SGD** was tested for its scalability and robustness, but it consumed the most time and delivered unstable results. **MLP** took longer to train than linear models but less time than SGD, and it achieved the best accuracy, making it the final model of choice.

Future projects would benefit from using a **paid Databricks tier or larger compute resources** to avoid memory bottlenecks and frequent session shutdowns. This would enable training on the **full dataset** rather than sampled subsets, improving both model accuracy and reliability. Additionally, implementing **distributed ML frameworks** (such as Spark MLlib or TensorFlow on Spark) could help scale more complex models without overwhelming resources. Finally, starting with simple, interpretable models

and progressively moving to advanced ones (like MLPs) proved effective, and this approach should be carried forward in similar large-scale projects.

8. Conclusion

This assignment was a valuable learning experience in working with large-scale data on Databricks. We learned how to load and sample big datasets, explore them through EDA, and build predictive models for estimating trip amounts. Beyond the technical tasks, the project highlighted practical challenges such as getting familiar with the Databricks platform, understanding its syntax, managing resources, and dealing with the logistics of running a project in the cloud. Overcoming these hurdles not only improved our technical skills but also gave us insight into how real-world data science workflows operate on a scale. The final product — a model capable of predicting trip amounts — reflects both the analytical techniques we applied and the lessons we gained in handling cloud-based big data projects.